Machine Learning: Practice Midterm

Solutions

1. Evaluating classifiers (20 points)

A charity trains African giant pouched rats ("HeroRATs") to detect tuberculosis (TB) via their impressive sense of smell. To make sure the rats are effective, each rat is tested on 1000 samples of sputum. Suppose a laboratory test shows that 100 of them come from patients with TB. The rat pauses at 80% of the TB samples and at 10% of the others. Treat "has TB" as the positive class.

1.1 [4 points] Write down the confusion matrix.

Solution: 100 TB samples and 900 non-TB samples. TP $= 0.8 \times 100$, FP $= 0.1 \times 900$.

	rat pauses	rat moves on
TB	TP = 80	FN = 20
no TB	FP = 90	TN = 810

1.2 [2 points] What are precision and recall for this rat?

Solution: Precision $= 80/(80+90) = 80/170 \approx 0.47$. Recall $= 80/100 = 0.80$.

1.3 [2 points] Do we care more about improving precision or recall for this problem, and why?

Solution: Recall. A missed TB patient (false negative) goes untreated and keeps infecting others; a false alarm (false positive) only costs one extra laboratory test.

The rat handler also records how long the rat pauses at each sample before moving on. Here are 10 samples, 5 from TB patients and 5 without TB:

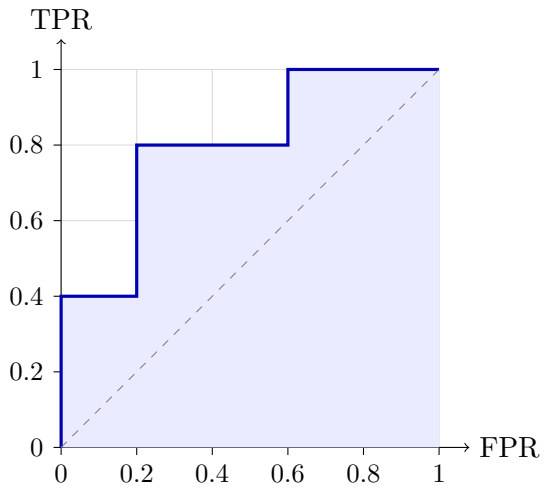
sample	A	B	C	D	E	F	G	H	I	J
pause (seconds)	2.6	0.6	4.6	2.0	1.3	3.7	4.2	1.0	3.0	2.4
TB?	yes	no	yes	no	yes	no	yes	no	yes	no

We say a rat flags a sample as TB if it pauses for over t seconds.

1.4 [10 points] Draw the ROC curve.

Solution: Sorted by pause, longest first: C (TB), G (TB), F (no), I (TB), A (TB), J (no), D (no), E (TB), H (no), B (no). As t passes each sample, a TB sample moves the point up by $1/5 = 0.2$ and a non-TB sample moves it right by 0.2. Starting at $(0, 0)$, the points are

$(0, 0.2)$, $(0, 0.4)$, $(0.2, 0.4)$, $(0.2, 0.6)$, $(0.2, 0.8)$, $(0.4, 0.8)$, $(0.6, 0.8)$, $(0.6, 1)$, $(0.8, 1)$, $(1, 1)$.



1.5 [2 points] Compute the AUC.

Solution: The area under the staircase, in columns of width 0.2 with heights $0.4, 0.8, 0.8, 1, 1$: $\text{AUC} = 0.2 \times (0.4 + 0.8 + 0.8 + 1 + 1) = 0.80$.

2. Support vector machines (20 points)

2.1 [10 points] A soft-margin SVM returned $\boldsymbol{w} = (4, 3)$, $b = -12$.

point	A	B	C	D	E
x_1	3	1	2.5	1	3
x_2	1	2	1	2.5	0.5
label	+	−	+	−	−

(a) Compute $f(\boldsymbol{x})$ and the predicted class of each point.

(b) What is the width of the margin?

(c) Compute each point's distance to the decision boundary.

(d) Which points are support vectors? Which point is on the margin, which is inside it, and which is misclassified?

Solution: $f(\boldsymbol{x}) = 4x_1 + 3x_2 - 12$ and $\|\boldsymbol{w}\| = \sqrt{4^2 + 3^2} = 5$.

point	A	B	C	D	E
$f(\boldsymbol{x})$	3	−2	1	−0.5	1.5
predicted class	+	−	+	−	+
distance $\lvert f(\boldsymbol{x})\rvert/\|\boldsymbol{w}\|$	0.6	0.4	0.2	0.1	0.3
$y\, f(\boldsymbol{x})$	3	2	1	0.5	−1.5

(b) Margin width $= 2/\|\boldsymbol{w}\| = 2/5 = 0.4$.

(d) The support vectors are the points with $y\, f(\boldsymbol{x}) \leq 1$: C, D and E. C is on the margin ($y\, f = 1$), D is inside the margin but correctly classified ($y\, f = 0.5$), and E is misclassified (label −, predicted +).

2.2 [6 points] Write down the objective function for the soft-margin SVM and explain why this is what we want to optimize.

Solution:

$$\min_{\boldsymbol{w},\, b,\, \boldsymbol{\xi}} \ \tfrac{1}{2}\|\boldsymbol{w}\|^2 + C\sum_i \xi_i \quad \text{subject to} \quad y_i(\boldsymbol{w}\cdot\boldsymbol{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

Minimizing $\frac{1}{2}\|\boldsymbol{w}\|^2$ maximizes the margin $2/\|\boldsymbol{w}\|$. The slack ξ_i measures how far point i falls short of being on the correct side, outside the margin; the sum penalizes these violations. C sets the trade-off between a wide margin and few violations.

2.3 [2 points] Suppose our soft-margin SVM obtains 95% training accuracy but 60% validation accuracy. What does this suggest about our hyperparameter C?

Solution: The large gap between training and validation accuracy means the model overfits. C is probably too large: violations are so expensive that the margin narrows to fit individual training points. Try a smaller C (wider margin, more slack), chosen on validation data.

2.4 [2 points] What are support vectors? Is every training datapoint that falls inside the margin a support vector?

Solution: Support vectors are the training points that determine the solution: the points on the margin, inside it, or misclassified, that is, those with $y_i f(\boldsymbol{x}_i) \leq 1$. Moving any other point (while it stays outside the margin) does not change the SVM. Yes: every point inside the margin is a support vector.

3. Nonlinear regression (20 points)

3.1 [4 points] A regression tree is trained on six points:

x	1	2	3	4	5	6
y	1	3	2	7	9	8

Consider the split $x < 3.5$. What does each side predict? What is the SSE of this split?

Solution: Left: the mean of $1, 3, 2$, which is 2. Right: the mean of $7, 9, 8$, which is 8. SSE $= (1 + 1 + 0) + (1 + 1 + 0) = 4$.

3.2 [6 points] What happens to the fit, toward overfitting or toward underfitting, as you increase (a) the maximum depth of a regression tree, (b) k in kNN regression, (c) ε in SVR? Explain each.

Solution: (a) More, smaller leaves: toward overfitting. (b) Each prediction averages more neighbors, so the fit is smoother: toward underfitting (at $k = n$ it predicts the overall mean everywhere). (c) A wider tube: more points cost nothing and the fit flattens: toward underfitting.

3.3 [5 points] In a training set the targets y range from 10 to 50. Can (a) a regression tree, (b) kNN regression, (c) SVR predict $y = 60$ for some new input? Explain.

Solution: (a) No: every leaf predicts an average of training y values, so its prediction stays between 10 and 50. (b) No, for the same reason: the prediction is an average of k training y values. (c) Yes: the prediction is $\boldsymbol{w}\cdot\boldsymbol{x} + b$, which keeps growing as $\boldsymbol{x}$ moves away from the data. (A kernel SVR can also go outside the training range; accept either explanation.)

3.4 [5 points] A model predicts crop yield from rainfall (in mm, 0 to 2,000) and average temperature (in °C, 10 to 35). We change rainfall to meters, so it now ranges from 0 to

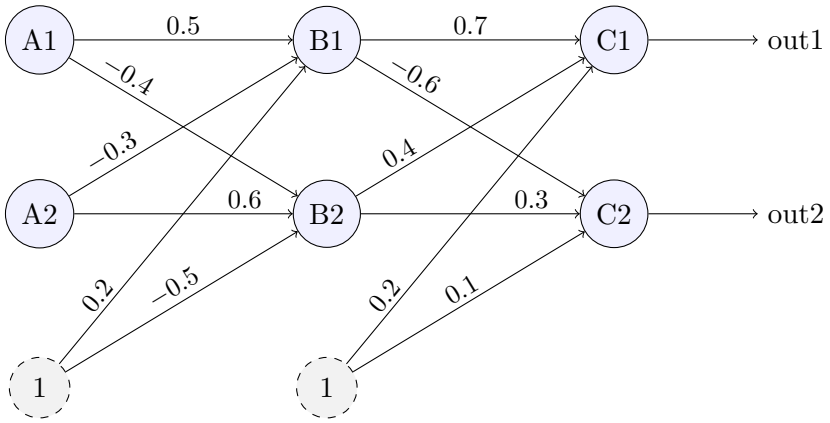
2. For each of a regression tree, kNN and SVR, do the predictions change? Why or why not?

Solution: Regression tree: no. A split looks at one feature at a time, so "rainfall < 800 mm" becomes "rainfall < 0.8 m" and every split's SSE is unchanged. kNN: yes. Distances now depend almost only on temperature, so different points become the nearest neighbors. SVR: yes. The penalty $\frac{1}{2}\|\boldsymbol{w}\|^2$ depends on the units of each feature, and with a kernel, distances change as for kNN.

4. Neural networks (30 points)

In this question, row j of a weight matrix $\boldsymbol{W}^{(l)}$ holds the weights into node j of layer l: $W_{ji}^{(l)}$ is the weight from node i of the previous layer to node j. The bias of node j is $b_j^{(l)}$, the weight on the input that is always 1.

4.1 [8 points] In the network below, out1 represents Class A and out2 represents Class B. All nodes use the sign function as their activation function: $\text{sign}(z) = +1$ if $z > 0$ and -1 if $z < 0$.



Show the forward pass, in matrix form, of the record with Feature1 $= 6$ and Feature2 $= 4$. How would this network classify the record (Class A or Class B)?

Solution: From the diagram:

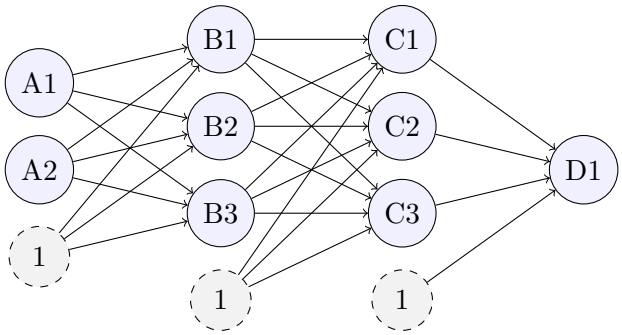
$$\boldsymbol{W}^{(1)} = \begin{pmatrix} 0.5 & -0.3 \\ -0.4 & 0.6 \end{pmatrix}, \quad \boldsymbol{b}^{(1)} = \begin{pmatrix} 0.2 \\ -0.5 \end{pmatrix}, \qquad \boldsymbol{W}^{(2)} = \begin{pmatrix} 0.7 & 0.4 \\ -0.6 & 0.3 \end{pmatrix}, \quad \boldsymbol{b}^{(2)} = \begin{pmatrix} 0.2 \\ 0.1 \end{pmatrix}.$$

$$\boldsymbol{z}_B = \boldsymbol{W}^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)} = \begin{pmatrix} 0.5 \cdot 6 - 0.3 \cdot 4 + 0.2 \\ -0.4 \cdot 6 + 0.6 \cdot 4 - 0.5 \end{pmatrix} = \begin{pmatrix} 2.0 \\ -0.5 \end{pmatrix}, \quad \boldsymbol{B} = \text{sign}(\boldsymbol{z}_B) = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

$$\boldsymbol{z}_C = \boldsymbol{W}^{(2)}\boldsymbol{B} + \boldsymbol{b}^{(2)} = \begin{pmatrix} 0.7 - 0.4 + 0.2 \\ -0.6 - 0.3 + 0.1 \end{pmatrix} = \begin{pmatrix} 0.5 \\ -0.8 \end{pmatrix}, \quad \boldsymbol{C} = \text{sign}(\boldsymbol{z}_C) = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

out1 $= +1$ and out2 $= -1$: the network predicts Class A.

4.2 [14 points] The network below has three layers of weights. A record with input $A_1 = 2$, $A_2 = 1$ and an actual value $Y = 4$ gets passed in.



	function	derivative
B layer: sigmoid	$f(z) = 1/(1+e^{-z})$	$f(z)\,(1-f(z))$
C layer: ReLU	$f(z) = 0$ if $z < 0$; $f(z) = z$ if $z \geq 0$	0 if $z < 0$; 1 if $z \geq 0$
D layer: linear	$f(z) = z$	1
loss: squared error	$E = \frac{1}{2}(Y' - Y)^2$	$\partial E/\partial Y' = Y' - Y$

Here Y' is the network's output, D1. The weights during the forward propagation are

$$\boldsymbol{W}^{(1)} = \begin{pmatrix} 0.5 & -1.0 \\ 1.0 & 0.5 \\ -0.5 & 1.5 \end{pmatrix}, \boldsymbol{b}^{(1)} = \begin{pmatrix} 0.5 \\ -1.0 \\ 0 \end{pmatrix}, \quad \boldsymbol{W}^{(2)} = \begin{pmatrix} -1.0 & 0.5 & -1.0 \\ 2.0 & 1.0 & -0.5 \\ 0.5 & 1.5 & 1.0 \end{pmatrix}, \boldsymbol{b}^{(2)} = \begin{pmatrix} 0.5 \\ -0.5 \\ -1.0 \end{pmatrix},$$

$$\boldsymbol{W}^{(3)} = \begin{pmatrix} 1.5 & -0.4 & 2.0 \end{pmatrix}, \quad b^{(3)} = 0.5.$$

You do not need to forward propagate. The node output values during the forward propagation are

A layer	B layer	C layer	D layer
$A_1 = 2$	$B_1 = 0.62$	$C_1 = 0$	$D_1 = 2.32$
$A_2 = 1$	$B_2 = 0.82$	$C_2 = 1.25$	
	$B_3 = 0.62$	$C_3 = 1.16$	

Adjust $W_{21}^{(1)}$, the weight from A1 to B2, using a learning rate of 1.

Solution: Work backwards from the loss, one layer at a time.

Output: $\partial E/\partial D_1 = D_1 - Y = 2.32 - 4 = -1.68$. D is linear, so this is also the derivative with respect to D1's input sum.

C layer: $\partial E/\partial z_{C_k} = -1.68 \cdot W_{1k}^{(3)} \cdot \mathrm{ReLU}'(z_{C_k})$. C1 outputs 0, so its input sum is negative and its slope is 0: C1 passes nothing back.

$$\boldsymbol{\delta}_C = (\,-1.68 \cdot 1.5 \cdot 0,\ \ -1.68 \cdot (-0.4) \cdot 1,\ \ -1.68 \cdot 2.0 \cdot 1\,) = (\,0,\ 0.672,\ -3.36\,).$$

B2 feeds all three C nodes, so add the three paths: $\partial E/\partial B_2 = \sum_k \delta_{C_k} W_{k2}^{(2)} = 0 \cdot 0.5 + 0.672 \cdot 1.0 + (-3.36) \cdot 1.5 = -4.368$.

Through the sigmoid: $\delta_{B_2} = -4.368 \cdot B_2(1 - B_2) = -4.368 \cdot 0.82 \cdot 0.18 = -0.6447$.

The weight: $\partial E/\partial W_{21}^{(1)} = \delta_{B_2} \cdot A_1 = -0.6447 \cdot 2 = -1.289$.

Update: $W_{21}^{(1)} \leftarrow 1.0 - 1 \cdot (-1.289) = 2.29$.

4.3 Short answers (8 points in all).

(a) [3 points] Suppose the output D1 used a sigmoid, the label were $Y \in \{0, 1\}$, and the loss were cross-entropy (log loss) instead of squared error. Which step of your backpropagation in 4.2 would change, and how?

Solution: Only the first step. With a sigmoid output and cross-entropy, the derivative of the loss with respect to D1's input sum is $p - Y$, where p is the sigmoid output (the derivative of the loss and the sigmoid's slope cancel to this). Everything after that is the same chain rule as before.

(b) [3 points] Why do the hidden layers need nonlinear activation functions? What would the network in 4.2 compute if every activation were linear?

Solution: A composition of linear functions is linear. With linear activations, D1 would be $\boldsymbol{w} \cdot \boldsymbol{A} + b$ for some $\boldsymbol{w}$ and b: the whole network would be one linear model, no matter how many layers it has.

(c) [2 points] While training a network, the training loss keeps going down, but the validation loss is lowest at epoch 40 and then rises. What is happening, and what would you do?

Solution: The network is overfitting after epoch 40. Use early stopping: keep the weights from the epoch with the lowest validation loss (epoch 40), and stop once it has not improved for a set number of epochs (the patience).